

COMP 3311

DATABASE MANAGEMENT SYSTEMS

LECTURE 21

QUERY PROCESSING: SIZE ESTIMATES; QUERY OPTIMIZATION

SIZE ESTIMATION

The result size of an operation determines
i) the cost of the operation and
ii) the cost of subsequent operations.

✎ Its accurate estimation is important for query processing cost estimation and for query optimization.

- To estimate the result size of an operation, the database catalog keeps various statistics about the sizes of data in the database.
- These statistics should be updated whenever the database is updated.

✎ This can incur substantial overhead.

- Consequently, DBMSs update their statistics only periodically.

✎ They are unlikely to be completely accurate but nevertheless are sufficient for reasonable size (and cost) estimation in most cases.

SIZE ESTIMATION: STATISTICS

- The **database catalog** (**data dictionary**) stores, for each relation r , the following statistics:

n_r the number of tuples in r .

B_r the number of pages containing tuples of r .

l_r the size of a tuple of r in bytes.

bf_r the blocking factor of r (i.e., the number of tuples of r that fit into one disk page).

$V(A, r)$ the number of **distinct values** of attribute A that appear in r .

➤ Equivalent to $\pi_A(r)$.

☞ If tuples of r are stored together physically in a file, then

$$B_r = \lceil n_r / bf_r \rceil.$$

HT_i the **number of levels** in the index i (i.e., the height of index i).

- For a balanced tree index (such as a B⁺-tree) on attribute A of relation r , $HT_i = \lceil \log_{f_i}(V(A, r)) \rceil$.
 - where f_i is the **average fan-out** of the internal nodes of the index.
 - $\min f_i = \lceil n/2 \rceil$ and $\max f_i = n$ for internal nodes where n is the maximum fan-out of an index node.
- For a hash index, HT_i is 1 (or 1.2 if there are overflow buckets).

LB_i the **number of lowest-level index pages** in i (i.e., the number of pages at the leaf level of the index).



SIZE ESTIMATION: SELECTION CARDINALITY AND SELECTIVITY

$SC(\theta, r)$ The selection cardinality, SC , of predicate θ for relation r , is the *average number of tuples that satisfy the predicate θ* .

$\text{Selectivity}(\theta, r) = SC(\theta, r) / n_r$ (i.e., the *fraction* of tuples that satisfy θ)

✎ Selectivity is a number between 0 and 1.

✎ The smaller the selectivity, the fewer tuples are selected and the smaller the result size of an operation.

SIZE ESTIMATION: EQUALITY SELECTION

Equality selection: $\sigma_{A=v}(r)$

$SC(A=v, r) = n_r / V(A, r) \Rightarrow$ number of r tuples / number of distinct A values in r

Selectivity($A=v, r$) = $1 / V(A, r) \Rightarrow$ 1 / number of distinct A values in r

✎ Assumes uniform distribution of values of A in r .

$\lceil SC(A=v, r) / bf_r \rceil \Rightarrow$ the number of pages that these tuples will occupy if they are ordered on attribute A .

- If the tuples are not ordered on A , then each tuple may reside in a different page.

✎ For equality selection on a candidate key attribute:

$SC(A=v, r) = 1$ (only 1 tuple will qualify)

Selectivity($A=v, r$) = $1 / n_r$

SIZE ESTIMATION: EQUALITY SELECTION EXAMPLE

Given **Sailor** with $n_{\text{Sailor}} = 40,000$ tuples in $B_{\text{Sailor}} = 500$ pages

```
select sailorld
from Sailor
where rating=8;
```

- If $V(\text{rating}, \text{Sailor}) = 10$ (i.e., 10 distinct rating values), then the condition $\text{rating}=8$ is expected to retrieve
 $SC(\text{rating}=8, \text{Sailor}) = n_{\text{Sailor}} / V(\text{rating}, \text{Sailor}) = 40,000 / 10 = \underline{4,000}$ tuples
 (i.e., $1/10$ of the tuples *assuming uniform distribution of rating values*).

- The estimated page I/O cost to execute the query using
 - a **clustering index** on rating is $1/10 * 500 = \underline{50}$ page I/Os.
 - a **non-clustering index** on rating is $\underline{4000}$ page I/Os since *each tuple* is assumed to require a page I/O.
 - In both cases we do not consider the cost of accessing the index because it is negligible. The index contains only 10 entries since there are only 10 rating values.
 - a **linear search** is $\underline{500}$ page I/Os.
 - a **binary search** (provided that the file is sorted on rating) is

$$\begin{aligned} \overline{\log_2(B_r)} + \overline{\text{binary search}} \frac{\text{\#pages of Sailor that qualify}}{SC(A, r)} - 1 &= \overline{\log_2(500)} + \overline{\text{1st page}} \frac{4000}{80} - 1 = 9 + 50 - 1 \\ &= \underline{58} \text{ page I/Os.} \end{aligned}$$

SIZE ESTIMATION: NON-EQUALITY SELECTIONS

$n_{\text{Sailor}}: 40,000$
 $B_{\text{Sailor}}: 500$
 $V(\text{rating}, \text{Sailor}) = 10$

Range selection: $\sigma_{A \leq v}(r)$
(The case for $\sigma_{A \geq v}(r)$ is symmetric.)

- Assume that $\min(A, r)$ and $\max(A, r)$ are available in the catalog.

$SC(A \leq v, r) = 0$ if $v < \min(A, r)$ (since no tuples will qualify)

$SC(A \leq v, r) = n_r$ if $v \geq \max(A, r)$ (since all tuples will qualify)

$SC(A \leq v, r) = n_r * \frac{v - \min(A, r)}{\max(A, r) - \min(A, r)}$ (assumes uniform distribution of tuples over the values)

Example:

Suppose $\min(\text{rating})=1$ and $\max(\text{rating})=10$.

For each rating there are $40,000/10=4,000$ tuples.

Then $SC(\text{rating} \leq 2, \text{Sailor}) = n_{\text{Sailor}} * (2 - 1) / (10 - 1)$
 $= 40,000 * 1/9 = \underline{4445}$ tuples (actual value 8000).

```
select sailorId
from Sailor
where rating <= 2;
```

👉 This is not a very good estimate for small ranges;
can get more accurate estimates using histograms.

HISTOGRAM

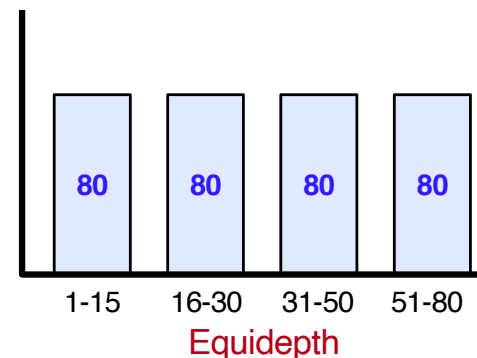
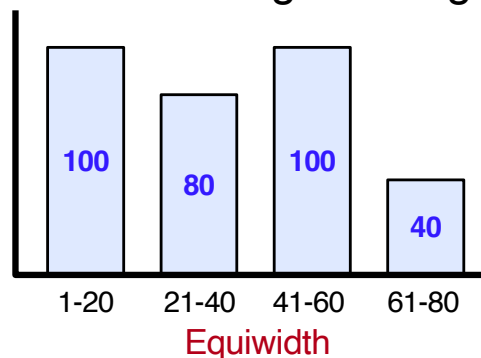
- The previous estimates assume that **each value of A has the same probability** (i.e., that the values of A are **uniformly distributed** over the tuples).

✋ **The uniformity assumption rarely holds in practice.**

- A **histogram assumes** values are **uniformly distributed within each partition** (i.e., local uniformity but not global uniformity).

Equiwidth divide A values into subranges of **equal size** (e.g., equal rating ranges).

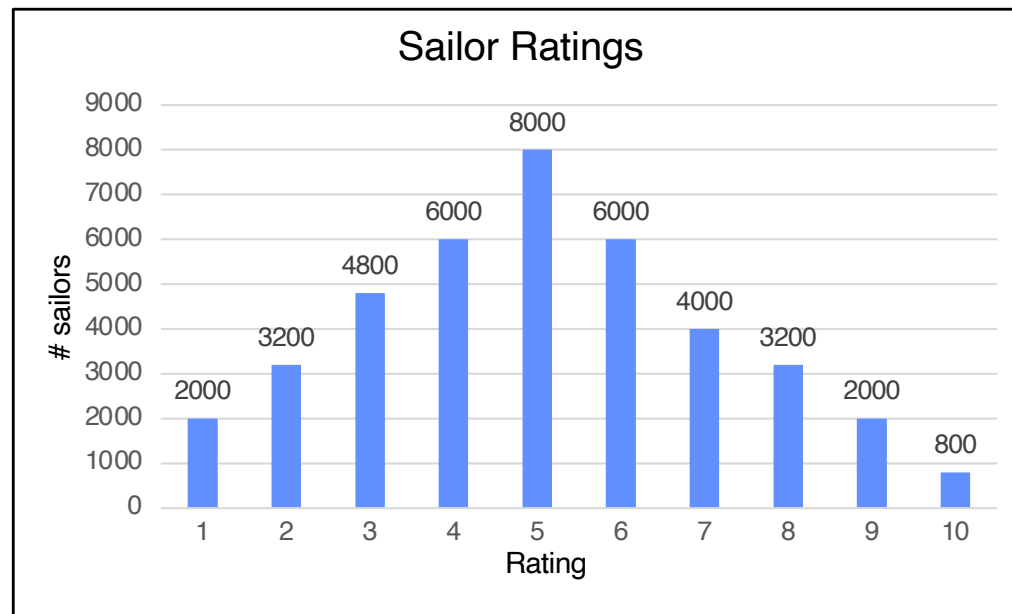
Equidepth divide A values into subranges such that the **number of tuples within each subrange is equal** (i.e., equal number of sailors in each rating subrange).



HISTOGRAM (CONTD)

Example: Given the histogram below on sailor ratings, then
 $SC(\text{rating} \leq 2, \text{Sailor}) = 2000 + 3200 = 5200$.
(Compared to 4445 under the uniform distribution assumption.)

```
select sailorId  
from Sailor  
where rating <= 2;
```



This is an
eqiwidth
histogram.

ATTRIBUTE INDEPENDENCE

- The previous estimates assume that **values of attributes are independent**.

👉 **The attribute independence assumption rarely holds in practice.**

Example: Consider the **Sailor** relation and **assume that the rating of a sailor increases with his/her experience**. We have histograms on both **rating** and **age**. Furthermore, the number of sailors with age 20 and 50 is equal.

- The two queries below are estimated to have the **same SC**.

```
select *  
from Sailor  
where rating=10 and age=20;
```

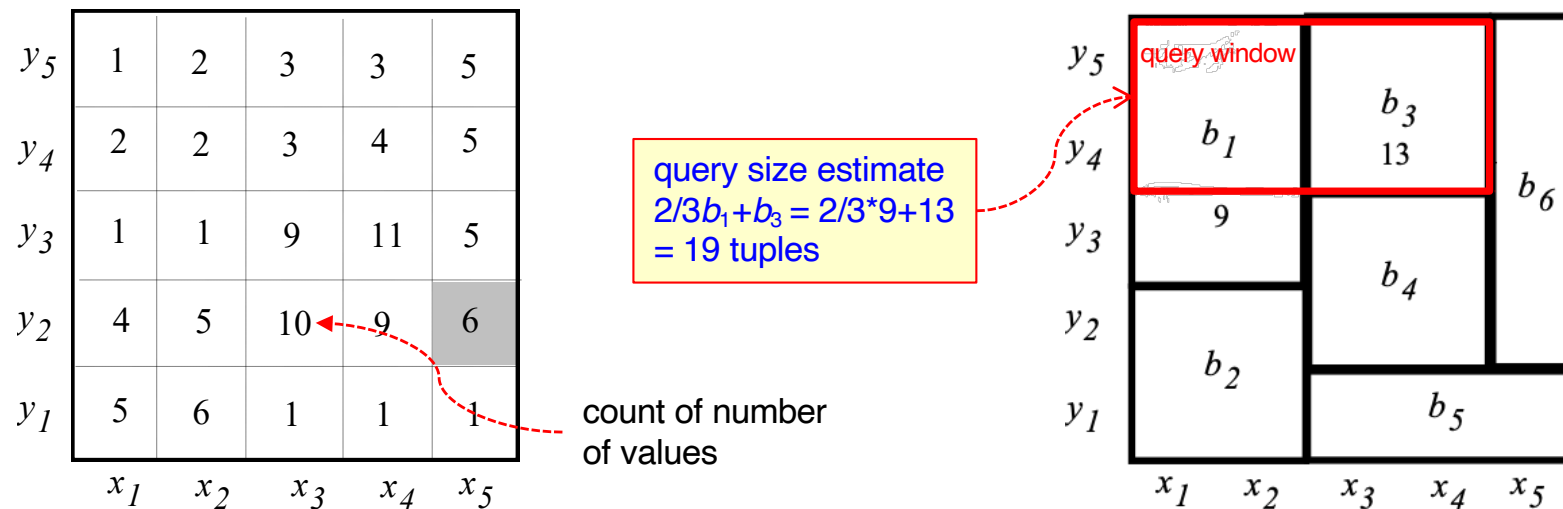
```
select *  
from Sailor  
where rating=10 and age=50;
```

Which query should be expected to retrieve more tuples?

Solution: Multidimensional histograms

MULTI-DIMENSIONAL HISTOGRAM

- Divide the search space (e.g., age-rating) into partitions, so that **data values in each partition** have an *almost uniform distribution of values*.
- Keep in the buffer the **partition extents** (i.e., the ranges of values included in the partition) and the **number of tuples per partition**.
- Use this information to estimate the number of tuples in the **query window**.



SIZE ESTIMATION: COMPLEX SELECTIONS

Conjunction: $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$

The probability that a tuple satisfies the conjunctive condition is the **product of the selectivities** of the predicates. If the **selectivity** of θ_1 is $\frac{s_1}{n_r}$, θ_2 is $\frac{s_2}{n_r}$... θ_i is $\frac{s_i}{n_r}$, then

$$SC(\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}, r) = n_r * \frac{s_1}{n_r} * \frac{s_2}{n_r} * \dots * \frac{s_n}{n_r} = n_r * \frac{s_1 * s_2 * \dots * s_n}{n_r^n}$$

Disjunction: $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$

The probability that a tuple satisfies the disjunctive condition is 1 minus the probability that it will satisfy *none* of the conditions. If the **selectivity** of θ_i is $\frac{s_i}{n_r}$, then $\overline{\theta_i}$ is $(1 - \frac{s_i}{n_r})$ and

$$SC(\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}, r) = n_r * (1 - (1 - \frac{s_1}{n_r}) * (1 - \frac{s_2}{n_r}) * \dots * (1 - \frac{s_n}{n_r}))$$

based on the logical equivalence: $\theta_1 \vee \theta_2 \vee \dots \vee \theta_n = \overline{\overline{\theta_1} \wedge \overline{\theta_2} \wedge \dots \wedge \overline{\theta_n}}$

SIZE ESTIMATION: JOIN

The Cartesian product $r \times s$ contains $n_r * n_s$ tuples.
Each tuple occupies $l_r + l_s$ bytes.

- If $r \cap s = \emptyset$ (i.e., r and s have no common attributes)
 - The number of tuples in $r \bowtie s$ is the same as that in $r \times s$ (i.e., $n_r * n_s$).
- If $r \cap s = A$ is a candidate key of r
 - Each tuple of s joins with at most one tuple from r (since the join attributes are a candidate key of r).

☞ The number of tuples in $r \bowtie s$ is no greater than number of tuples in s .

Example: In $\text{Sailor} \bowtie \text{Reserves}$, sailorId in Sailor is a candidate key of Sailor .

☞ The result has exactly n_{Reserves} tuples.

☞ If sailorId in Reserves can be null, then the number of tuples in the result will be less than the number of tuples in Reserves .

SIZE ESTIMATION: JOIN (CONTD)

The Cartesian product $r \times s$ contains $n_r * n_s$ tuples.
Each tuple occupies $l_r + l_s$ bytes.

- If $r \cap s = A$ is a (not null) foreign key in s referencing r
 - The number of tuples in $r \bowtie s$ is the same as the number of tuples in s (since the join attribute A is a foreign key in s referencing r).
 - 👉 The case where $r \cap s$ is a foreign key in r referencing s is symmetric.

Example: Estimate the size result for $\text{Sailor} \bowtie \text{Reserves}$ where the join attribute, sailorld , in Reserves is a foreign key of Sailor .

Catalog information: $n_{\text{Sailor}} = 10,000$ $n_{\text{Reserves}} = 5,000$

- Since sailorld is a foreign key in Reserves , the size of the result is the same as $n_{\text{Reserves}} = \underline{5,000}$ (i.e., every tuple in Reserves joins with a tuple in Sailor).

SIZE ESTIMATION: JOIN (CONTD)

The Cartesian product $r \times s$ contains $n_r * n_s$ tuples.
Each tuple occupies $l_r + l_s$ bytes.

- If $r \cap s = A$ is not a key for r or s
 - If we assume that every tuple in r produces $SC(A, s) = n_s / V(A, s)$ tuples in $r \bowtie s$ (i.e., each r tuple joins with $n_s / V(A, s)$ s tuples), then the number of tuples in $r \bowtie s$ is estimated to be:

$$n_r * \frac{n_s}{V(A, s)}$$

- If we assume that every tuple in s produces $SC(A, r) = n_r / V(A, r)$ tuples in $r \bowtie s$ (i.e., each s tuple joins with $n_r / V(A, r)$ r tuples), then the number of tuples in $r \bowtie s$ is estimated to be:

$$n_s * \frac{n_r}{V(A, r)}$$

👉 The lower of these two estimates is probably the more accurate one.

SIZE ESTIMATION: JOIN (CONTD)

Example: Estimate the result size of $\text{Sailor} \bowtie \text{Reserves}$ *without using information about foreign keys*.

Catalog information:

$$n_{\text{Sailor}} = 10,000 \quad V(\text{sailorId}, \text{Sailor}) = 10,000 \quad V(\text{sailorId}, \text{Reserves}) = 2,500$$

$$n_{\text{Reserves}} = 5,000 \quad \Rightarrow \text{only 2,500 sailors have reservations}$$

- The two estimates are:

$$n_{\text{Reserves}} * \frac{n_{\text{Sailor}}}{V(\text{sailorId}, \text{Sailor})} = 5,000 * 10,000 / 10,000 = \underline{5,000}$$

$$n_{\text{Sailor}} * \frac{n_{\text{Reserves}}}{V(\text{sailorId}, \text{Reserves})} = 10,000 * 5,000 / 2,500 = \underline{20,000}$$

- We **choose the lower estimate**, which, in this case, is the same as our earlier computation using foreign keys.

SIZE ESTIMATION: OTHER OPERATIONS

Projection: $\pi_A(r)$

- The estimated size is $V(A, r)$ (i.e., one tuple for each distinct A value).

Aggregate operations: group by A

- The estimated size is $V(A, r)$ (i.e., one tuple for each distinct A value).

Set operations: $r \cup s$, $r \cap s$, $r - s$

- For union / intersection of selections **on the same relation**, rewrite the query and use size estimate for selections.
 - E.g., $\sigma_{\theta_1}(r) \cup \sigma_{\theta_2}(r)$ can be rewritten as $\sigma_{\theta_1 \vee \theta_2}(r)$.
- For operations **on different relations**:
 - Estimated size of $r \cup s$ = size of r + size of s .
 - Estimated size of $r \cap s$ = minimum (size of r , size of s).
 - Estimated size of $r - s$ = r .

👉 **All three estimates may be quite inaccurate, but provide upper bounds on the sizes.**

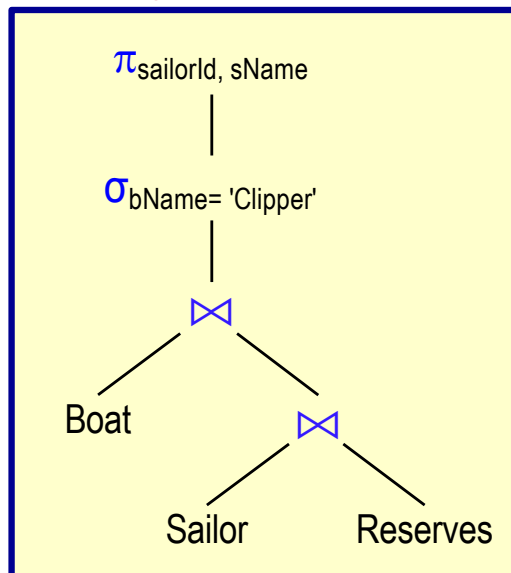
QUERY OPTIMIZATION: OVERVIEW

Query optimization is the process of selecting the most efficient query-evaluation plan from among many strategies.

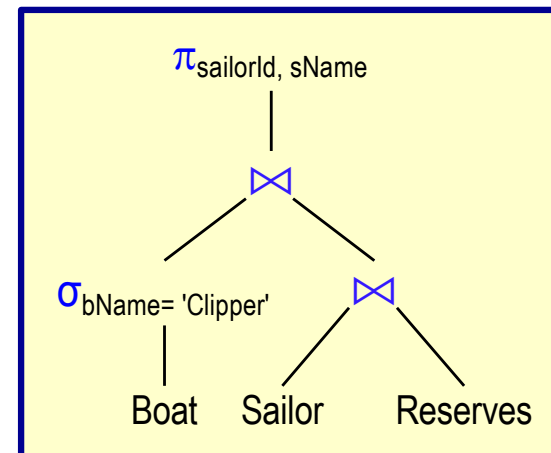
There may be several alternative ways of evaluating a given query.

- Equivalent expressions: Find the ids and names of sailors who have reserved the boat named Clipper.
- Different algorithms could be used for each operation.

Query Evaluation Tree



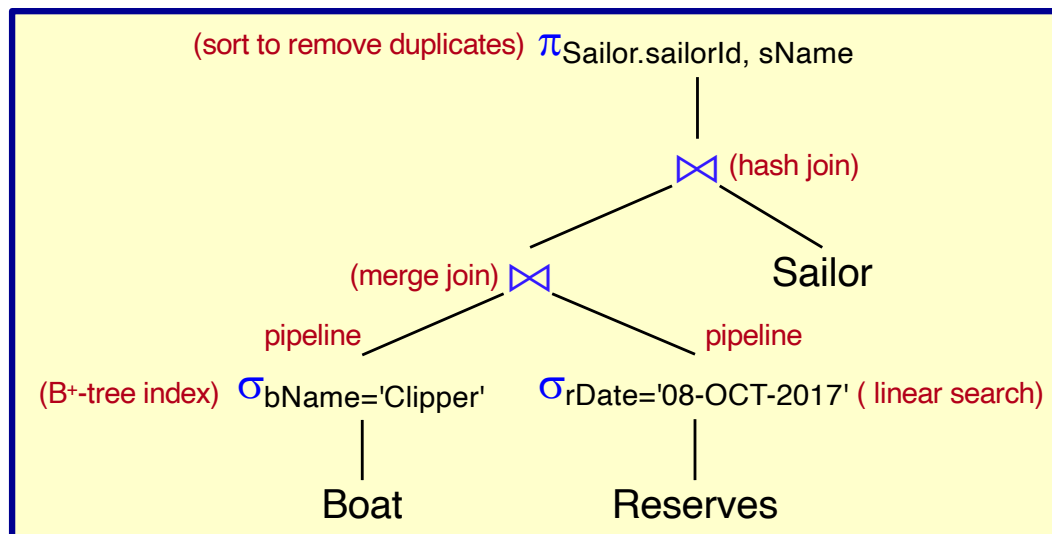
Query Evaluation Tree



QUERY OPTIMIZATION: OVERVIEW (CONTD)

- An **evaluation plan** defines exactly what algorithm is used for each operation, and how the **execution** of the operations is coordinated.
- ☞ **The cost difference between evaluation plans can be enormous (e.g., seconds versus days in some cases).**

Query Evaluation Tree



Most DBMSs allow you to view the evaluation plans for a query.

QUERY OPTIMIZATION: OVERVIEW (CONTD)

Cost-Based Optimization

1. Generate **logically equivalent evaluation plans** using equivalence rules for relational algebra expressions.
 2. Estimate the **cost of each plan**.
 3. Execute the plan with the **minimum expected cost**.
- Estimation of the plan cost is based on:
 - **Statistical information** from the database.
 - The number of tuples, number of distinct values for an attribute, etc.
 - **Statistics estimation** for intermediate results.
 - To compute the cost of complex expressions.
 - **Cost formulae for algorithms**, which are computed using statistics.

QUERY OPTIMIZATION: OVERVIEW (CONTD)

Heuristic Optimization

1. Perform **first operations** that reduce subsequent costs $\Rightarrow$ **perform selections early**.
 - Reduces the number of tuples.
2. **Remove unneeded attributes** $\Rightarrow$ **perform projections early**.
 - Reduces the number of attributes (i.e., the size of each tuple).
3. Try to **utilize existing indexes**.
 - **Perform *most restrictive* selection and join operations before** other similar operations.
- The goal is to **transform the relational algebra tree** by **using a set of rules** that typically (but not in all cases) improves execution performance.

👉 **Some DBMSs use only heuristics while others combine heuristics with partial cost-based optimization.**

TRANSFORMATION OF RELATIONAL EXPRESSIONS

- Two relational algebra expressions are **equivalent** if they *generate the same set of tuples* on *every legal* database instance.
 - The order of the tuples generated is irrelevant.
 - We do not care if they generate different results on databases that violate integrity constraints.
- **SQL** inputs and outputs are **multisets of tuples** (i.e., contain duplicates).
 - Two expressions in the multiset version of the relational algebra are equivalent if they generate the same multiset of tuples on every legal database instance.
- An **equivalence rule** states that two forms of an expression are equivalent.

 **Can replace an expression of the first form by one of the second, or vice versa.**

EQUIVALENCE RULES: SINGLE OPERATIONS

Cascading of projections: $\pi_{a_1}(R) \equiv \pi_{a_1}(\pi_{a_2} \dots (\pi_{a_n}(R)) \dots)$ where $a_1 \subseteq a_2 \dots \subseteq a_n$

Example: $\pi_{sName}(\text{Sailor}) \equiv \pi_{sName}(\pi_{sName, sailorId}(\text{Sailor}))$

➤ Allows early removal of unwanted attributes to reduce intermediate result size.

Cascading of selections: $\sigma_{c_1 \wedge c_2 \dots \wedge c_n}(R) \equiv \sigma_{c_1}(\sigma_{c_2} \dots (\sigma_{c_n}(R)) \dots)$

Example: $\sigma_{age=20 \wedge rating>7}(\text{Sailor}) \equiv \sigma_{rating>7}(\sigma_{age=20}(\text{Sailor}))$

Commutativity of selections: $\sigma_{c_1}(\sigma_{c_2}(R)) \equiv \sigma_{c_2}(\sigma_{c_1}(R))$

Example: $\sigma_{rating>7}(\sigma_{age=20}(\text{Sailor})) \equiv \sigma_{age=20}(\sigma_{rating>7}(\text{Sailor}))$

➤ Allows early removal of unwanted tuples to reduce intermediate result size.

Commutativity of joins / Cartesian products: $R \bowtie S \equiv S \bowtie R$

Associativity of joins / Cartesian products: $(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$

➤ Allows joins to be reordered to reduce intermediate result size.



EQUIVALENCE RULES: MULTIPLE OPERATIONS

Commutativity of selections with projections: $\pi_a(\sigma_c(R)) \equiv \sigma_c(\pi_a(R))$

👉 **Applicable only if the selection condition includes only attributes of the projection list.**

Legal Example: $\pi_{sName, age}(\sigma_{age=20}(\text{Sailor})) \equiv \sigma_{age=20}(\pi_{sName, age}(\text{Sailor}))$ ($age \in sName, age$)

Illegal Example: $\pi_{sName, rating}(\sigma_{age=20}(\text{Sailor})) \not\equiv \sigma_{age=20}(\pi_{sName, rating}(\text{Sailor}))$ ($age \notin sName, rating$)

➤ Remove unwanted attributes first or unwanted tuples first whichever reduces more the intermediate result size.

Commutativity of selections with joins / Cartesian products:

$$\sigma_c(R \text{ Join } S) \equiv (\sigma_c R) \text{ Join } S$$

👉 **Applicable if c involves only attributes of R .**

Legal Examples:

$\sigma_{sName='Joe' \wedge rating=7}(\text{Sailor Join Reserves}) \equiv (\sigma_{sName='Joe' \wedge rating=7} \text{Sailor}) \text{ Join Reserves}$

$\sigma_{sName='Joe' \wedge boatId=100}(\text{Sailor Join Reserves}) \equiv (\sigma_{sName='Joe'} \text{Sailor}) \text{ Join } (\sigma_{boatId=100} \text{Reserves})$

➤ Do selections first to reduce intermediate result size.

EQUIVALENCE RULES: MULTIPLE OPERATIONS

(CONT'D)

Projection Distributes Over Join: $\pi_a(R \text{ Join } S) \equiv (\pi_{a_1}(R) \text{ Join } \pi_{a_2}(S))$

– where

- i. a_1 is the subset of a that belongs to R *plus* the join attribute,
- ii. a_2 is the subset of a that belongs to S *plus* the join attribute.

Example: $\pi_{sName, rDate}(\text{Sailor Join Reserves}) \equiv$
 $\pi_{sName, rDate}((\pi_{sName, sailorId} \text{Sailor}) \text{ Join } (\pi_{rDate, sailorId} \text{Reserves}))$

➤ Allows unwanted attributes to be removed before doing the join to
reduce intermediate result size.

- If the join attribute is included in the list of projection attributes a , then:
 $\pi_a(R \text{ Join } S) \equiv (\pi_{a_1}(R) \text{ Join } \pi_{a_2}(S))$.

Example: $\pi_{sailorId, rDate}(\text{Sailor Join Reserves}) \equiv (\pi_{sailorId} \text{Sailor}) \text{ Join } (\pi_{sailorId, rDate} \text{Reserves})$

➤ Reduces the size of the join inputs and thus reduces intermediate result size.

QUERY PROCESSING: SUMMARY

Operation Processing

Selection: linear search; binary search; index lookup.

Sorting: external sort-merge if too large to fit into the buffer

Projection: sorting

Join: block nested-loop; indexed nested-loop; merge join; hash join

Expression evaluation: materialization; pipelining.

Query Processing

1. Transform an **SQL** query into a relational algebra expression.
2. Generate different query execution plans and evaluates their cost based on the cost and result size of each operation using statistics about the database.
3. Select an optimal query execution plan using heuristics and/or cost estimates.